

Construí el mismo MVP dos veces con 5 agentes IA

La segunda vez me costó 57% menos, 73% menos tiempo, y necesitó 75% menos intervenciones mías.

V1	V2
3h · \$4.21 · 12 manos	49 min · \$1.80 · 3 manos

PetDesk — recepcionista IA para peluquerías caninas.

MVP web: chat de reservas + dashboard del groomer + *backfill* automático de cancelaciones. Construido con 5 agentes IA trabajando en paralelo, orquestados como un tablero Kanban.

STACK

ORQUESTADOR	Multica self-host (32k ★, Docker)
MODELOS	Kimi K2.6 + DeepSeek V3.2 vía OpenRouter
CLI AGENTE	OpenCode
REPO + CI	GitHub + GitHub Actions
DEPLOY	Vercel (auto-deploy en push)

12 errores. *3 horas. \$4.21.*

EL ERROR MÁS CARO

"Le pedí al primer agente que scaffoldeara Next.js con `create-next-app`. El comando hizo `git init` fresco y me borró el remote a GitHub, el plan del repo, y todo el contexto. 5 minutos, \$0.40 quemados, 0 PRs."

Lección: los scaffolders agresivos no se mezclan con repos preexistentes. **El arquitecto scaffoldea; los agentes construyen encima.**

Las 3 lecciones que cambiaron todo.

i.

El arquitecto define contratos y scaffoldea.

El agente nunca toca esos archivos. Lo creativo va a él; lo mecánico, a ti.

ii.

"No toques X" no es contrato — es una sugerencia que el LLM ignora cuando le conviene.

Necesitas enforcement estructural: CODEOWNERS, branch protection, tests que fallen si lo viola.

iii.

Los costos reales son 3 a 5× las estimaciones single-call.

Por el re-envío de contexto en agentes con muchos tool calls. Mide en OpenRouter, no estimes.

Lo que metí en el template antes de invocar un solo agente.

01 **Template completo**

Scaffold + configs + CI + .gitignore agresivo + ESLint tuneado. Los agentes ya no scaffoldean nada.

02 **Contratos como tests ejecutables**

Funciones property-based con fast-check. Si la implementación del agente no las pasa, CI rojo. No es markdown — es enforcement real.

03 **CONSTITUTION.md + CODEOWNERS**

Reglas duras + bloqueo estructural sobre `src/domain/`. Sin OpenSpec ni Spec-Kit — los probé y no encajan.

04 **Cost routing por agente**

Frontend en DeepSeek V3.2 (4× más barato). Lógica + QA en Kimi K2.6. Calidad indistinguible para UI mecánica.

Spec ejecutable, no markdown.

Una suite property-based parametrizada. Cualquier implementación de la interfaz debe pasarla — o CI rojo.

```
// tests/contracts/SchedulerPort.contract.ts
export function schedulerPortContract(makeSut: ()⇒ Factory) {
  describe('SchedulerPort', ()⇒ {
    it('cancel devuelve candidates FIFO por createdAt');
    it('cancel filtra candidates por serviceId');
    it('cancel filtra por ventana (fuera del slot NO matchea)');
  });
}

// tests/engine.test.ts (escrito por el agente Dev Motor)
schedulerPortContract(()⇒ ({
  scheduler: new Scheduler(repo, clock), repo, clock
})));
```

El test es el contrato. El agente no puede "interpretarlo" mal — si su implementación no pasa, no se mergea.

Mismo producto, mismos modelos. Distinto proceso.

MÉTRICA	V 1	V 2	Δ
Costo OpenRouter	\$4.21	\$1.80	-57%
Tiempo de reloj	~3 h	49 min	-73%
Intervenciones humanas	~12	3	-75%
Tests al cierre	18	31	+72%
Scope violations	2	0	✓
PRs limpios al 1er intento	3/4	4/4	✓

Empieza por el template.

El resto se ordena solo: contratos + CONSTITUTION + cost routing hacen el 80% del trabajo de disciplina.

V1 - EL CAOS	github.com/rubenaros/petdesk
V2 - EL TEMPLATE	github.com/rubenaros/petdesk-v2
DOCS + SCRIPTS	github.com/rubenaros/AgentCode

El experimento completo, los 12 errores y los snippets reales → [Medium](#) (link en el post).